



Navalha 2

Migração paralela para JUCE/C++

Conceitos, histórico, arquitetura, funcionalidades, fluxo completo, procedimentos realizados, validações e direção de interface.

Referência funcional: Navalha 2 v0.28.1

Referência técnica: workspace NAVALHA2 e navalha_app_project_v2_v0.28.1.zip

Data do relatório: 30 de julho de 2026

Estado estimado da transposição: 98,5%, sem substituir o runtime original

Sumário executivo

O Navalha 2 é um instrumento de performance e composição baseado em seleção, corte e recombinação de materiais sonoros. Seu fluxo enfatiza a relação entre preparação do áudio e decisão performática: carregar fontes, definir regiões, criar slices, organizar padrões, transformar o material, tocar, gravar e devolver a gravação ao próprio instrumento.

A versão v0.28.1 distribui responsabilidades entre interface HTML/CSS/JavaScript, bridge Python, patches Pure Data e WebAudio. A migração paralela busca concentrar áudio, tempo musical, gravação, projeto e render offline em um núcleo C++ determinístico, mantendo a aplicação atual como referência até que a paridade esteja demonstrada.

Princípio de segurança: durante toda a migração, as pastas app, bridge e core foram preservadas. O desenvolvimento ocorreu dentro de juce, com builds limitados a dois jobs, temporários removidos e auditorias periódicas de espaço em disco.

Dimensão	Estado
Núcleo musical e DSP C++	Implementado e coberto por regressões
Standalone JUCE Linux	Compila, abre e reproduz áudio real
Projetos v1/v2 e portable pack	Codec, migração, inspeção e render implementados
Gravação e MASTER	Implementados; validações objetivas e gravação real realizadas
Interface final	Em reavaliação conjunta; nenhum layout foi declarado definitivo
Aprovação de substituição	Depende de testes humanos finais, hardware, dois monitores e decisão de licença

1. Conceito do instrumento

1.1 Material sonoro como matéria de performance

O Navalha não trata o áudio apenas como uma faixa linear. Cada fonte pode ser delimitada por uma região e repartida em unidades performáveis. Essas unidades podem ser selecionadas, alternadas entre SOURCE A e SOURCE B, reordenadas, protegidas em MEMORY e submetidas a transformações controladas.

1.2 Continuidade entre criação e registro

Um dos conceitos centrais é o ciclo:

performar → gravar → recarregar → cortar → recombinar → performar novamente

A gravação não é somente uma saída final. Ela pode tornar-se novo material, criando uma prática iterativa de composição e resample.

1.3 Agência humana e assistência

O Assisted Performer foi projetado como coparticipante limitado. O humano mantém autoridade sobre PLAY, STOP e REC. A automação toma decisões delimitadas por frase, seed, faixa de BPM, vocabulário permitido e intensidade, podendo atuar sobre padrões, região, cortes, pitch, fragmentos e mix. MEMORY, KEEP, RESTORE e locks preservam espaços de decisão humana.

1.4 Identidade Arcade

A identidade aprovada combina superfícies quase pretas, amarelo para ação e seleção, aço para controles neutros e vermelho reservado à gravação e erros. O mascote original, o wordmark pixelado branco/amarelo e os ícones aprovados permanecem ativos. A paleta documentada usa, entre outras, as cores #080a0a, #101313, #171b1b, #ffd84a, #fff08a, #78828a e #e01820.

2. Histórico resumido

Período/versão	Evolução principal
Origem, 2009	NAVALHA Arcade, associado a Glerm Soares; instrumento baseado em Pure Data e performance com fragmentos.
Reestruturação 2026	Interface web, bridge Python, organização modular dos patches, projetos, biblioteca, tutorial e gravação.
v0.15–v0.16	Layout desktop fixo: biblioteca/log à esquerda; PREPARE, waveform e PERFORM no corpo; TIME/PITCH/OUTPUT/SLICES na base.
v0.17.0	Project v2 e Portable Pack v2 com SOURCE A/B independentes, slices, padrões, MEMORY, TRACE e estado performático.
v0.17.2–v0.17.3	REC simplificado, identidade aprovada, histórico de takes e ciclo TAKE → SOURCE A/B.
v0.17.4–v0.17.5	Identidade NAVALHA Arcade e STOP global, encerrando sequencer, players, tarefas e loops pendentes.
v0.17.6	Histórico persistente de gravações e reconstrução segura dos takes.
v0.18+	Assisted Performer, autonomia delimitada, FORM, TRACE, mixer, vozes e direção de álbum/masterização.
v0.28.1	Referência funcional mais recente, preservada no ZIP canônico indicado pelo usuário.
Migração JUCE/C++	Implementação paralela de motor, ferramentas, standalone e validações, sem apagar o sistema anterior.

3. Arquitetura de referência v0.28.1

```
HTML/CSS/JavaScript | interface, waveform, navegação e automação | WebAudio  
complementar/MASTER | HTTP e eventos | Bridge Python | arquivos, cache,  
projetos, metadados e histórico | UDP/FUDI | Pure Data Vanilla | playback,  
slices, sequencer, pitch, mixer, vozes e REC
```

Essa arquitetura tornou possível evoluir rapidamente, mas possui vários pontos de sincronização: navegador, servidor local, bridge, porta UDP, patches e dispositivo de áudio. A migração não é uma tradução mecânica; ela exige reproduzir regras musicais, temporais e sonoras.

4. Arquitetura JUCE/C++ proposta

```
Interface JUCE nativa | comandos SPSC sem locks SessionModel + AudioEngine  
C++ | sequencer sample-accurate | players/slices A e B | gestos, FORM,  
TRACE e Assisted | mixer, Heritage Pitch e MASTER | telemetria atômica |  
FIFO pós-MASTER | dispositivo de áudio + writer WAV em thread separada 0  
mesmo núcleo atende render offline, testes e ferramentas CLI.
```

O callback não acessa disco, não processa JSON, não espera locks e não faz alocações por amostra. Estruturas de capacidade fixa, buffers imutáveis, comandos enfileirados e telemetria atômica protegem o caminho realtime.

5. Funcionalidades transpostas

5.1 Fontes, regiões e slices

- duas fontes estéreo independentes, A e B;
- bancos entre 1 e 128 slices;
- divisão em 4, 8, 16, 32 ou 64;
- START/END, BLADE, undo, MICRO e reverse;
- arquivos originais imutáveis;
- waveform estéreo com cache min/max e sobreposição dos limites.

5.2 Sequencer e performance

- dez padrões de oito células;
- códigos A0-A127, B0-B127 e GAP;
- GRID, FREE e JITTER com seed reproduzível;
- STOP como fronteira global, invalidando agendamentos pendentes;
- STUTTER ×4, BURST ×8, MICRO e REVERSE sample-accurate;
- MEMORY e transformações MUTATION, EROSION e DECONSTRUCT reversíveis.

5.3 FORM, TRACE e Assisted

- FORM Director com cinco cenas padrão e até 16 cenas;
- edição de transição, bancos, energia, densidade, tensão, estabilidade, continuidade, contraste e movimento estéreo;
- TRACE XY de até 512 pontos e reprodução em loop por amostras;
- Assisted Performer determinístico, com RNG Mulberry32 compatível com JavaScript;
- decisões por frase sobre source, pattern, região, cortes, pitch, fragmentos e mix;
- AUTO MIX conservador, limitado a balance, pan e width.

5.4 Caminho de áudio

SOURCE A/B players → SOURCE MIX → Heritage Pitch → MASTER → recorder/output

- mixer A/B com level, pan, width, mute, solo e balance;
- rampas de 15 ms e envelopes de declick;
- duas vozes virtuais com source, divisão, padrão, foco, pitch, envelope, level e pan;
- Heritage Pitch com duas cabeças, interpolação de quatro pontos, janelas cossenoidais e crossfade dry/processado;
- MASTER suavizado compartilhado por realtime e offline.

5.5 Projeto, portable e gravação

- Project v1 legível e migração para o modelo v2;
- Project v2 preservando sources, slices, patterns, mixer, FORM, TRACE, MEMORY, Assisted e seeds;
- portable pack ZIP com CRC32, allowlist, deduplicação e defesa contra path traversal;
- REC pós-MASTER em PCM16, PCM24 ou float32;
- writer em thread separada, drenagem controlada e publicação atômica;
- reserva mínima de 1 GiB, limite de uma hora e teto RIFF;
- metadados RIFF LIST/INFO.

5.6 TRACK MASTER e ALBUM MASTER

- análise de peak, RMS, LUFS estimado, crest, correlação e headroom;
- TRACK MASTER com EQ, dinâmica, saturação, width e ceiling;
- receita JSON interoperável com a interface web;
- ALBUM MASTER com trims, fades, gaps, manifesto e batch PCM24;
- preflight, limites e publicação por arquivos parciais.

6. Fluxo completo do usuário

1. **Preparar:** abrir SOURCE A e/ou B, observar waveform, escolher região, dividir ou criar cortes.

2. **Programar:** selecionar pattern e preencher oito passos com slices A/B ou GAP.
3. **Performar:** escolher GRID/FREE/JITTER, ajustar BPM, disparar gestos, pitch, TRACE e transformações.
4. **Dirigir:** opcionalmente usar FORM e Assisted dentro de limites definidos pelo músico.
5. **Misturar:** ajustar level, pan, width, balance, mute/solo, vozes e MASTER.
6. **Gravar:** escolher formato, iniciar REC pós-MASTER, acompanhar frames/drops e finalizar.
7. **Persistir:** salvar Project v2 leve ou Portable Pack com áudio associado.
8. **Reprocessar:** carregar um take como nova fonte e reiniciar o ciclo de corte e recombinação.
9. **Finalizar:** analisar/renderizar TRACK MASTER e organizar ALBUM MASTER quando aplicável.

7. Procedimentos realizados na migração

7.1 Preservação e estratégia

- criação da árvore paralela juce;
- nenhuma remoção ou renomeação do runtime v0.28.1;
- JUCE 8.0.13 e CMake mantidos como dependências locais, fora das fontes;
- builds limitados a dois processos;
- artefatos temporários gerados em /tmp e removidos.

7.2 Construção incremental

A transposição começou pelas estruturas sem dependência gráfica: slices, padrões, sequencer, mixer, players e render offline. Em seguida foram integrados AudioEngine, filas realtime, gravação, projetos, portable packs, MASTER, FORM, TRACE, Assisted e, por último, o shell JUCE nativo.

7.3 Compatibilidade

Foram implementados codecs JSON próprios com limites de tamanho e profundidade, migração v1, modelo v2, referências de áudio, portable ZIP e ferramentas read-only. O inspetor de projetos prova que a serialização canônica v2 é idempotente e rejeita arquivos truncados ou versões futuras não suportadas.

7.4 Ferramentas criadas

Ferramenta	Finalidade
navalha_compare_wav	RMS, erro, correlação, SNR e compensação de latência.
navalha_render_portable	Portable ZIP → PCM24 offline.
navalha_render_pitch_fixture	Fixture para Heritage Pitch.
navalha_analyze_master	Métricas objetivas de um WAV.

navalha_render_master	TRACK MASTER PCM24 atômico.
navalha_inspect_album	Validação e planejamento de manifesto.
navalha_render_album	Batch ALBUM MASTER.
navalha_inspect_project	Inspeção read-only de Project v1/v2.

8. Validações executadas

Validação	Resultado
Build standalone JUCE Linux	Aprovado
CTest completo	9/9 testes aprovados
Stress DSP combinado	30 s recorrentes; soak virtual de 600 s em blocos 64/511, resultado idêntico
Sanitizers	AddressSanitizer e UndefinedBehaviorSanitizer sem achados
LeakSanitizer	Não contabilizado: incompatível com o supervisor ptrace atual
Gravação virtual	PCM24, 60 s, 2.880.000 frames, zero drops, WAV reaberto e removido
MASTER C++ x WebAudio	353.708 frames; 0,176 dB peak, 0,136 LUFS e 0,0009 de diferença de correlação
Project v1/v2	Fixtures, migração, idempotência e rejeição adversarial aprovadas
Teste gráfico real	Aplicativo abriu; SOURCE carregou; waveform e áudio funcionaram
Transporte/timing	PLAY/STOP, GRID/FREE/JITTER testados pelo usuário sem anomalia relatada
Mixer A/B	Balance, pan, width, mute e solo testados pelo usuário
Heritage Pitch operacional	Teste auditivo de operação relatado como normal
REC físico	PCM24 estéreo, 44,1 kHz, 61,2 s, 2.698.752 frames

A gravação real analisada atingiu 0 dBFS repetidamente. O arquivo estava estruturalmente correto, mas indicou necessidade de novo passe com mais headroom. Isso não invalida o writer; permanece uma questão de ganho, monitoramento e decisão de proteção/limiter no fluxo performático.

9. Interface e layout

O layout ainda não é definitivo. A v0.28.1 é a referência funcional; a v0.17.5 é uma referência particularmente clara de hierarquia: biblioteca e log à esquerda, waveform grande, PREPARE, PERFORM, gestos e quatro cartões inferiores. O mockup mais recente mantém essa superfície principal e desloca MIX, FORM, ASSISTED e MASTER para navegação complementar.

Diretrizes acordadas:

- preservar mascote, favicon, wordmark e cabeçalho aprovados;
- waveform como maior objeto visual;
- PLAY, STOP e REC imediatamente reconhecíveis;
- uma tela em 1920×1080; rolagem somente como fallback;
- não declarar um protótipo como layout final sem avaliação conjunta;
- evitar que funcionalidades avançadas escondam a prática básica.

10. Segurança, armazenamento e integridade

- reserva de 1 GiB durante REC;
- limites de WAV, render e manifests;
- publicação atômica e limpeza de .partial;
- defesa contra traversal em ZIP e caminhos portáteis;
- fontes originais nunca modificadas;
- última auditoria: aproximadamente 453 MiB de build, 282 MiB de dependências e 1,1 TiB livre no volume;
- nenhum temporário pesado deixado pelas validações descritas.

11. Pendências e critérios de substituição

1. concluir a reavaliação conjunta do layout;
2. comparação auditiva formal A/B do Heritage Pitch contra Pure Data;
3. comparação auditiva cega do MASTER contra WebAudio;
4. gravação física prolongada sem xruns e com headroom adequado;
5. abrir, salvar e reabrir projetos reais do usuário v1/v2;
6. teste real em dois monitores com uma única autoridade de áudio/estado;
7. decisão registrada sobre AGPLv3/licença comercial e distribuição JUCE;
8. empacotamento e instalação final para os sistemas alvo.

A estimativa de 98,5% representa implementação técnica aproximada, não autorização para substituir a v0.28.1. A referência atual deve continuar disponível até que os critérios humanos, operacionais e jurídicos sejam concluídos.

12. Próxima etapa recomendada

Produzir duas ou três variações de layout de baixa ambiguidade, avaliá-las com o usuário em 1920×1080 e somente então consolidar a interface JUCE. Em paralelo, executar os testes auditivos formais e usar um projeto real do usuário para encerrar a validação de persistência.

13. Referências do workspace

- ANALISE_MIGRACAO_JUCE_CPP.txt
- navalha_app_project_v2_v0.28.1.zip
- docs/DEVELOPMENT_LOG.md
- docs/NAVALHA_ARCADE_v0.17.4.md
- docs/FIXED_LAYOUT_v0.15.9.md
- docs/STANDARD_LAYOUT_v0.16.0.md
- docs/LAYOUT_PERFORMER_v0.19.0.md
- juce/README.md
- juce/docs/PARIDADE_V0281.md
- juce/docs/MASTER_OBJECTIVE_COMPARISON.md
- juce/docs/FINAL_ACCEPTANCE_CHECKLIST.md

Documento vivo. Deve ser atualizado após cada decisão conjunta de layout e após os testes finais de hardware, persistência e audição.